

A WALKTHROUGH FOR STUDENTS

How an AI coding agent actually works

Inside **llmc-code** — a coding assistant that runs on your own laptop. No cloud. No API bill. We open every part and explain it in plain words.

~13,000 lines of Python

26 modules

1,355 tests

3 libraries

100% offline

Every slide answers three things — WHY we needed it · WHAT it is · HOW it works

01 The problem we are solving

WHY THIS EXISTS

You can download a great model today. You still can't use it to code.

A language model does exactly one thing: **you give it text, it gives you text back.** That's all. It cannot open a file. It cannot run your tests. It forgets everything the moment it replies.

So when you load a model in LM Studio, you get a **chat box**. What you actually wanted was something that reads your project, changes your code, runs your tests, sees what broke, and fixes it.

BUT RUNNING LOCALLY CHANGES EVERYTHING

A cloud tool can assume a huge, fast, well-behaved model. **We can't assume any of that.** Our model is smaller, slower, and messier. Almost every custom part in this deck exists because of that one difference.

THE MISSING PIECE

Something has to sit **between the model and your computer**. It gives the model a way to ask for things, does those things for it, and shows it what happened.

That middle program is called a **harness**. Building one is what this deck is about.

Why run it on your own machine?

- **Your code stays yours.** Nothing is uploaded anywhere.
- **It's free to use.** No per-word charges, no limits.
- **It works with no internet.** On a plane, in a lab, anywhere.

Five things local models force you to build

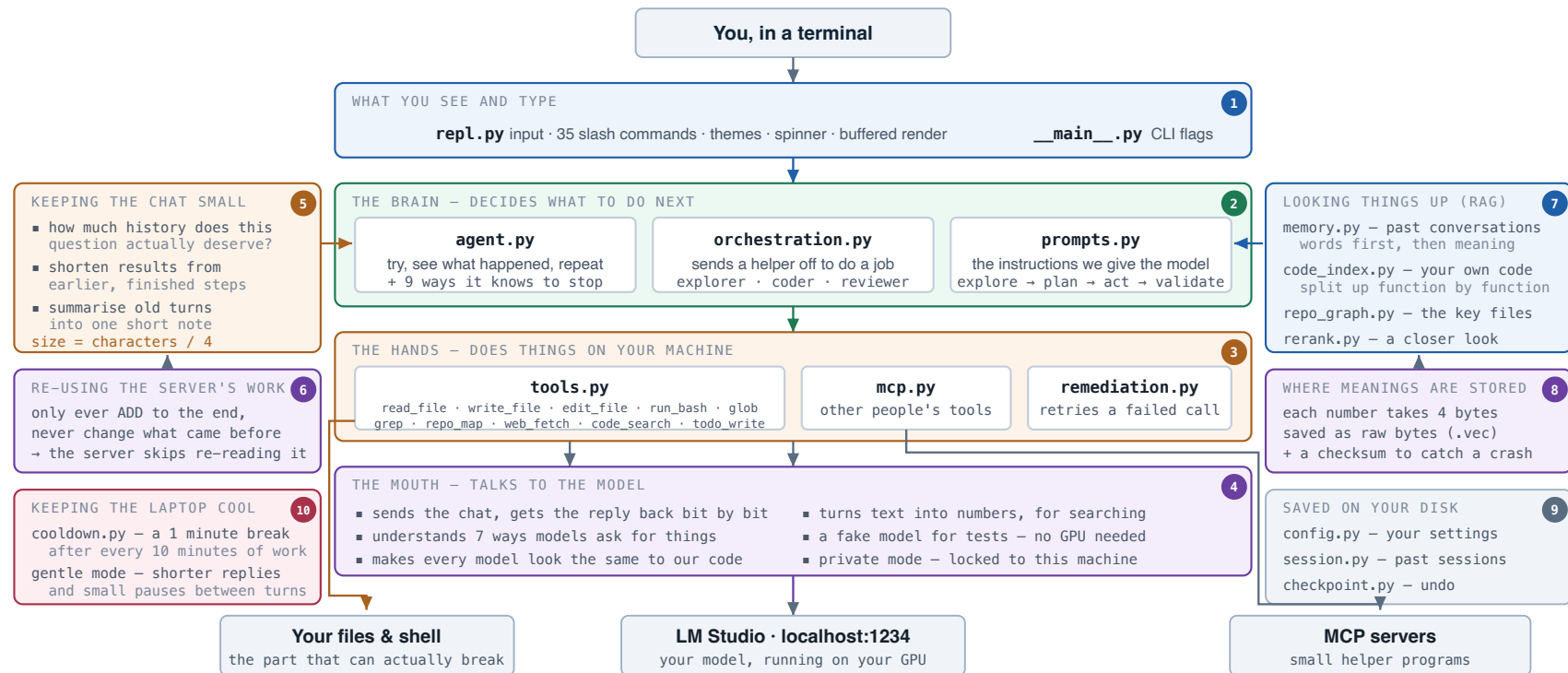
THE DIFFICULTY	WHAT WE HAD TO BUILD
Every model asks for things differently	7 different readers for the same request, plus a layer that fixes wrong spellings of the same option name
Long chats get very slow the model re-reads everything each time	Context management — keep the conversation small, and re-use the work the server already did
The model can only hold a little text at once	Search — store everything, then look up only the few bits this question needs
Laptops get hot	Forced breaks and shorter replies , so an hour of work doesn't cook the machine
It changes your real files	Ask-before-you-act, undo, and limits on where it can write

IN ONE SENTENCE

We are building the program that turns "a model on your laptop" into "an assistant that gets work done on your laptop."

02 The complete architecture

THE MAP · BOX NUMBERS BELOW



WHY IT IS BUILT IN LAYERS

The brain **never touches your files or the network itself**. It can only ask the hands and the mouth to do it. That one rule is what lets us swap the real model for a fake one and test the whole thing without a GPU.

WHAT THE SIDE BOXES DO

Box 3 shrinks the chat before sending it. **Box 4** finds only the bits this question needs. A cloud tool needs neither — both exist purely because this runs on your own machine.

HOW TO READ IT

Follow the arrows **top to bottom** for one question. **Each of the six numbered boxes gets its own slide next** — what it holds, how it works, and why it is built that way.

03 The one data structure everything else operates on

START HERE

Before any module makes sense, you need this. The assistant's entire "memory" is **one Python list of dictionaries**, in a variable called `agent.messages`.

No database, no hidden state, no framework object. Just a list. Every feature in this deck is an **operation on that list**.

`agent.messages` — the only five shapes that ever go in it

```
[
  {"role": "system",    "content": "You are a coding agent..."},    # [0], never moves

  {"role": "user",      "content": "fix the failing test"},

  {"role": "assistant", "content": "",                             # it wants something
   "tool_calls": [{"id": "call_1",
                     "function": {"name": "run_bash",
                                   "arguments": '{"command": "pytest -q"}' } ]},

  {"role": "tool",      "tool_call_id": "call_1",                  # the result
   "content": '{"ok": true, "exit": 1, "stdout": "FAILED test_parse.py..."}'},

  {"role": "assistant", "content": "The parser crashes on empty input..."}
]
```

TWO INVARIANTS YOU MUST NEVER BREAK

- **Index 0 is always the system message.** Every operation starts from index 1.
- **Every `tool_calls` entry needs a matching `tool` entry with the same `id`.** One without the other and the server rejects the *whole* request. Most agent bugs are this bug.

How fast does it grow?

One thing you type produces **1 + 2×N** entries, where N is how many actions the model took. "Fix the failing test" — run tests, read two files, edit, re-run — is **11 entries**, and those tool results can be thousands of lines.

Where does it actually live?

In RAM, as an attribute on the `Agent` object. **On disk**, rewritten after every completed turn:

`~/.llmcode/sessions/<project-id>.json`

```
{
  "cwd":      "/Users/you/projects/app",
  "updated_at": "2026-09-18T09:14:02+00:00",
  "model":    "qwen3-coder-30b",
  "title":    "fix the failing test",
  "messages": [ ... the exact list above ... ]
}
```

Four operations, and nothing else

Every technique later in this deck is one of these:

- **Append** to the end — all the loop ever does
- **Shorten** one entry's `content`, in place — the trim (box 5)
- **Replace** a run of old entries with one summary — compaction (box 5)
- **Copy** the list and add to the copy — recall and caching (boxes 6, 7)

WHY THIS SLIDE COMES FIRST

"Managing context" sounds abstract until you know it means **editing strings inside a Python list**. That is all it is.

04 Where everything is actually stored

ONE FOLDER, SEVEN FILES

Every piece of state this program keeps lives under **one folder in your home directory**. No database, no server, no cache daemon. You can open all of it in a text editor.

the whole state of the tool – chmod 0700, owner only

```
~/llmcode/
├── config.json           # your settings
├── mcp.json              # external tool programs
├── sessions/
│   ├── app-3f9a1c2b7e04.json      # THE CONVERSATION
│   ├── app-3f9a1c2b7e04.memory.json # past Q&A, for recall
│   ├── app-3f9a1c2b7e04.memory.json.vec # their vectors (binary)
│   └── app-3f9a1c2b7e04.code_index.json # your code, chunked
├── checkpoints/app-3f9a1c2b7e04/sessions/<id>/
│   ├── index.json             # what was backed up, when
│   └── blobs/*.blob           # the file bytes, for undo
```

What is that `app-3f9a1c2b7e04` part?

A **project id**, built from the folder you launched in:

```
project_id = "app" + "-" + sha256("/Users/you/projects/app")[:12]
             # ↑ folder name      ↑ first 12 hex of the FULL path
```

The name makes it readable. The hash of the *absolute path* makes it unique — so two different projects both called `app` never collide, and the same project always maps back to the same files.

Why one file per concern, not one big one?

FILE	WRITTEN WHEN
<code><proj>.json</code> the conversation	After every completed turn . Whole file rewritten.
<code><proj>.memory.json</code> past questions + answers	When a turn finishes, one record is added.
<code>...memory.json.vec</code> the numbers	Only when something new gets embedded.
<code><proj>.code_index.json</code> your code, in chunks	When a search runs and a file has changed.
<code>blobs/*.blob</code> undo copies	Just before each file edit.

Splitting them means a big write to one never risks the others, and the **numbers stay out of the readable files**.

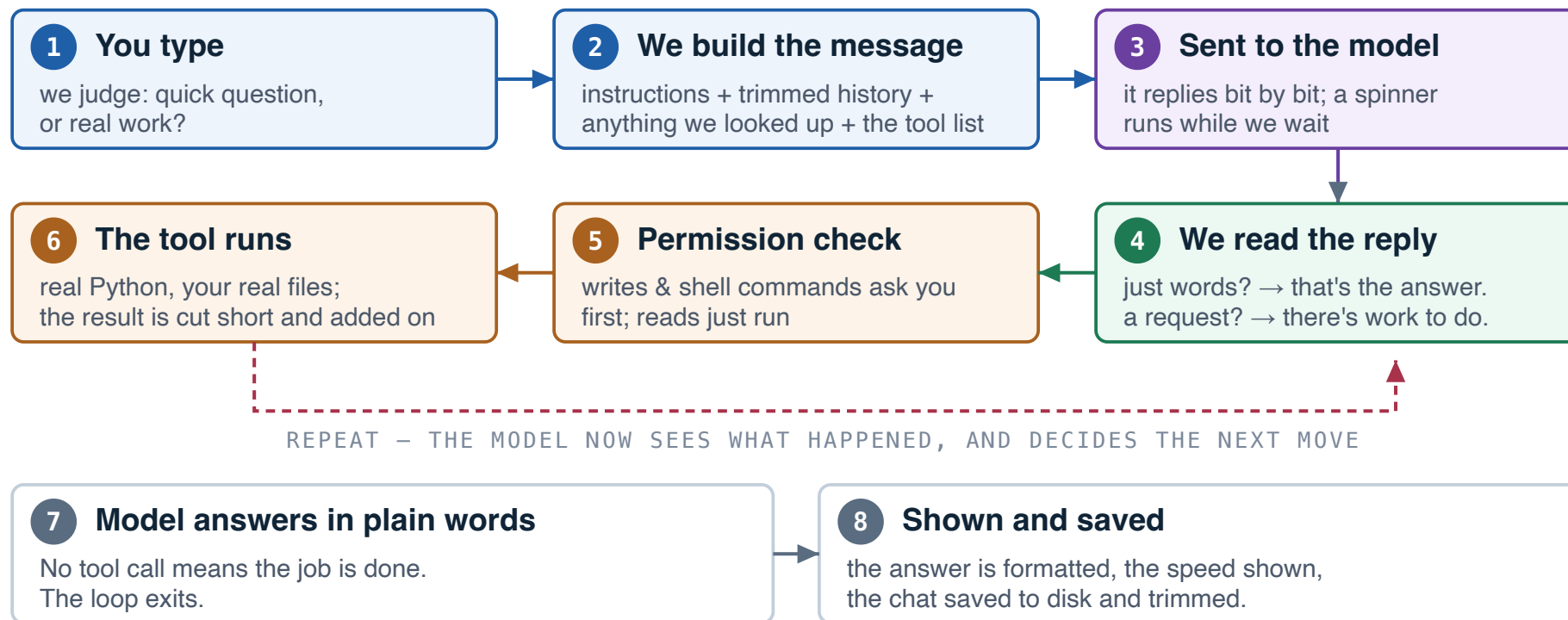
EVERY SINGLE WRITE USES THE SAME THREE STEPS

Write to a **temporary file in the same folder** → flush it to the disk → **rename** it over the real file.

Renaming is one indivisible operation, so a crash leaves you with **either the old file or the new one** — never a half-written one. Same three lines in `config.py`, `session.py`, `memory.py` and `checkpoint.py`.

WHAT THIS BUYS YOU AS A BUILDER

When something goes wrong you can `cat` the file and see exactly what the model was sent. Debugging an agent is mostly **reading the conversation you actually sent** — make that easy on day one.

**WHY IT LOOPS**

One question and one answer cannot fix a bug. The model has to **look, try something, see what happened, then try again**. Every time round, it knows more than it did before.

WHAT MAKES IT STOP

An answer with **no requests in it**. If the model just talks, it means it has nothing left to look up — so what it said *is* the answer, and we stop.

HOW IT STAYS UNDER CONTROL

A hard limit of **80 times round**, plus eight more stop-conditions. Anything that writes a file or runs a command **asks you first**. Just reading? It goes ahead.

06 The whole agent, in 20 lines

Take away 3,000 lines of safety checks and polish, and this is the whole thing. It is not pseudo-code — this runs, and we tested it with a fake model.

the heart of every coding assistant ever built

```
def agent(task, max_steps=12):
    messages = [{"role": "system", "content": SYSTEM},
                {"role": "user", "content": task}]

    for _ in range(max_steps):          # the iteration guard
        reply = client.chat.completions.create(
            model=MODEL, messages=messages, tools=SCHEMA
        ).choices[0].message
        messages.append(reply.model_dump(exclude_none=True))

        if not reply.tool_calls:        # plain words = DONE
            return reply.content

        for call in reply.tool_calls:
            args = json.loads(call.function.arguments)
            if call.function.name in NEEDS_OK and not ask_user():
                out = "DENIED by the user."
            else:
                try:
                    out = TOOLS[call.function.name](**args)
                except Exception as e:    # errors are DATA
                    out = f"ERROR: {type(e).__name__}: {e}"
            messages.append({"role": "tool",
                            "tool_call_id": call.id,
                            "content": str(out)[:4000]}) # ALWAYS cap

    return "Stopped: hit the step limit."
```

Five small decisions, each a big deal

THE LINE	WHY IT HAS TO BE THERE
<code>str(out)[:4000]</code>	Tool output is added to the conversation and re-sent forever after. One giant result ruins the rest of the session. So we cut it short right where it is produced .
<code>except → "ERROR: ..."</code>	If a tool crashes, the whole thing stops. Instead we turn the failure into a message — and the model reads it and tries something else.
<code>range(max_steps)</code>	Without a limit, a confused model keeps trying forever. Your laptop runs all night.
<code>NEEDS_OK</code>	Anything that could break something asks you first. You stay in charge.
The system prompt	We change how it behaves by writing English , not code. Adding "look before you answer" genuinely changes the quality of the result.

THE ERROR MESSAGE IS THE IMPORTANT ONE

Compare two ways of reporting the same failure:

- "error" → the model has nothing to go on. It guesses, and keeps guessing wrong.
- "path not found; did you mean llmcode/agent.py?" → fixed on the very next try.

Your error messages are now instructions to the model.

Write them for the model to read, not for a log file nobody opens.

The whole "brain" is one object with these fields

llmcode/agent.py — the state that matters

```

self.messages      # the list from slide 3 — the ONLY real state
self.provider      # who to send it to (real model, or the fake one)
self.registry      # {name: Tool} — what it is allowed to do
self.tool_names    # which of those to offer THIS agent
self.max_iterations# 80 — hard stop
self.memory        # the recall store (box 7), or None
self.context_budget# 12_000 — the target size for the list

# per-turn scratch, all reset at the top of run():
self._read_bytes   # how much it has read this turn
self._changed_files # what it edited, for the verify step
self.last_turn_details # full args+results, for Ctrl+0

```

Note what is **not** there: no file handles, no sockets. The brain can only call `self.provider` and things in `self.registry` — which is what lets the whole decision layer be tested with a fake model and a temp folder.

Two different agents, same class

A sub-agent is **the same Agent class**, constructed with a different system prompt and a smaller `registry`. An explorer literally has no `write_file` key in its dict — it cannot be talked into writing, because the function isn't reachable.

What one pass of the loop does, in order

#	STEP	TOUCHES
1	Cancelled? Cooldown due?	checked <i>before</i> anything is appended
2	Over budget? → summarise	<code>self.messages</code> (box 5)
3	Build the request	a copy : <code>self.messages + extras</code>
4	Stream the reply	<code>self.provider</code> (box 4)
5	No tool calls? → return	this is the only exit
6	Append <code>assistant</code> + run each tool + append <code>tool</code>	<code>self.registry</code> (box 3)
7	Check the loop guards	counters, below

The guards are just counters on local variables

```

consecutive_parse_errors # 3 all-bad rounds → stop
_consecutive_dup_batches # 4 identical rounds → stop
nudge_count              # max_nudges = 2 (it said nothing)
tool_markup_nudged       # one-shot latch
truncation_retried       # once per pass (reply got cut off)
for _iter in range(self.max_iterations): # 80, the outer stop

```

THE RULE THAT STOPS GUARDS MISFIRING

A round where **one call worked and one failed** is still progress — so the counter only increases when the **entire** round failed, and any good call resets it to zero.

Our first version counted *any* failure, and killed sessions that were working fine.

A tool is a five-field dataclass. That's the whole abstraction.

llmcode/tools.py

```
@dataclass
class Tool:
    name: str
    description: str          # the model READS this — it's a prompt
    parameters: dict          # JSON Schema for the arguments
    fn: Callable[[dict], dict]# the Python that runs
    requires_confirmation: bool = False

REGISTRY: dict[str, Tool] = {}          # name → Tool

def register(tool): REGISTRY[tool.name] = tool
```

Tools add themselves to `REGISTRY` when the module is imported. An agent is then given a **subset** of that dict — which is how role-scoping works.

The round trip, with the real names

```
1. openai_schema(names, registry)    # {name, description,
   → sent with the messages          # parameters} per tool
2. model replies: "run_bash", '{"command":"pytest"}'
3. normalize_tool_args(name, args)    # cmd→command, file_path→path
4. _permission_decision(tool, args)   # allow / ask / block
5. checkpoint.snapshot([path])       # before any write
6. result = tool.fn(args)             # ordinary Python
7. messages.append({"role":"tool",
   "tool_call_id": id, "content": json.dumps(result)[:cap]})
```

Every tool returns the same two shapes — and never raises

```
{"ok": True, "result": "..."}
{"ok": False, "error": "no such path: llmcode/agent.py"}
```

An exception ends your turn. A returned error becomes a `role:"tool"` entry the model **reads next pass** and corrects — which is why error text is written for the model, not for a log file.

The caps, and where they are enforced

LIMIT	APPLIES TO
<code>_MAX_OUTPUT</code> = 24,000 bytes	every tool result, the hard ceiling
250 lines / 10 MB scan guard	a whole-file <code>read_file</code>
500 matches / 100 matches	<code>glob</code> / <code>grep</code>
256 KB , streamed via <code>select()</code>	<code>run_bash</code> output
12,000 bytes	<code>code_search</code> — tighter on purpose

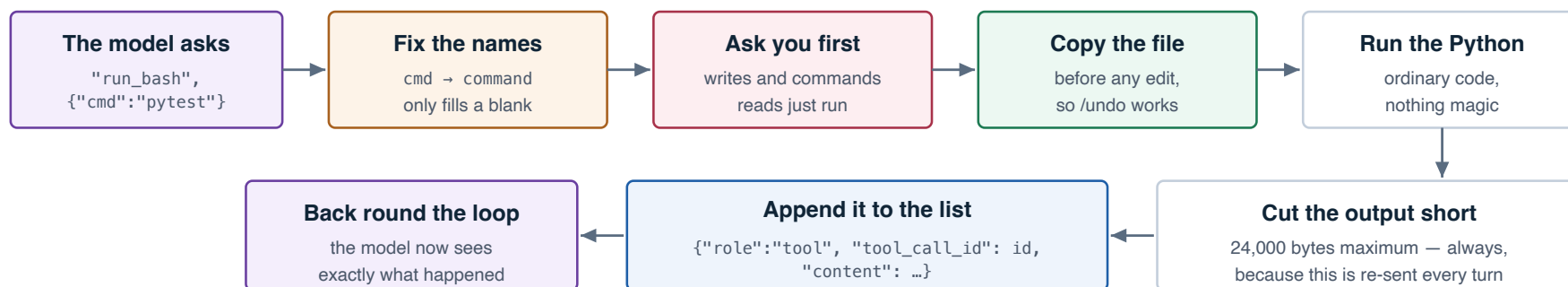
Caps live **inside the tool**. Whatever it returns is appended to the list and re-sent forever after — so the only safe place to stop it is where it is produced.

TWO DETAILS WORTH STEALING

- `run_bash` streams output through `select()` instead of waiting for the process to end — so a runaway log can't fill your RAM before the timeout fires. On timeout the whole **process group** is killed.
- `run_bash` truncates **head + tail**, while `read_file` truncates head-only — because a test failure summary is at the *bottom*.

09 One tool call, from request to result

WHAT HAPPENS BETWEEN THE MODEL ASKING AND THE MODEL SEEING



IF IT FAILS

The error is data, not a crash

```
{"ok": false, "error": "no such path: llmcode/agent.py"}
```

A crash would end your turn. An error gets read on the next pass, and corrected.

WHY THE ORDER MATTERS

Names are fixed **before** you are asked — so the prompt shows what will really run.

The file is copied **before** it is touched — undo cannot be added afterwards.

The output is cut **here**, at the source — nowhere else can stop a runaway result.

WHY SO MANY STEPS

The model is **untrusted input**. Every step between "it asked" and "it ran" is a chance to catch a mistake before it reaches your disk.

WHAT THE MODEL SEES

Only the last box: a `role: "tool"` entry carrying the capped result and the matching id. It never learns about the name-fixing or the file copy.

HOW IT RECOVERS

A failure is appended as an ordinary result. The next pass round the loop, the model reads the error and tries something else — **no crash, no lost turn**.

The contract: three event types, nothing else

Whatever model you loaded, this box yields a stream of plain dicts. The brain is written against *these*, so it never learns which model you're running.

llmcode/providers.py — every provider yields only these

```
{ "type": "text",      "text": "... a fragment of the reply ..."
  "type": "tool_call", "id", "name", "arguments": {... a dict ...}
  "type": "done",     "finish_reason", "output_tokens" }
```

Rules: arguments arrive already parsed into a dict, tool calls all come before `done`, and there is **exactly one** `done`, always last.

The fiddly part: reassembling a request from fragments

The reply streams in. One request's arguments arrive split across dozens of fragments, and two simultaneous requests arrive **interleaved** — told apart only by an index number.

llmcode/providers.py — the accumulation loop

```
for tc in delta.tool_calls:
    idx = tc.index or 0          # which call this belongs to
    slot = tool_buf.setdefault(idx, {"id": None, "name": None, "args": []})
    if tc.id:                    slot["id"] = tc.id
    if tc.function.name:         slot["name"] = tc.function.name
    if tc.function.arguments:
        slot["args"].append(tc.function.arguments) # collect...

# ...and only when the stream ends:
args = json.loads(''.join(slot["args"]))          # ...join ONCE
```

Fragments go into a **list**, joined once. Gluing strings one at a time re-copies the whole thing each fragment — with a 2,000-line file arriving in pieces, that alone is the difference between instant and painfully slow.

Why seven parsers live here

Many local models never use the proper channel. They **type the request out as text** and expect the client to spot it. Each family invented its own syntax:

FAMILY	WHAT ARRIVES AS PLAIN TEXT
proper channel	a real <code>tool_calls</code> field — easy
DeepSeek	<code>< tool_call_begin ></code> then JSON
Qwen · Hermes	<code><tool_call>{"name":...}</tool_call></code>
Mistral	<code>[TOOL_CALLS][{...}]</code>
Llama 3.x	<code><function=name>{...}</code> or <code>< python_tag ></code>
Qwen XML	<code><parameter=path>main.py</parameter></code>
our fallback	a fenced <code>json</code> block we taught it to emit

WHY THE XML ONE IS DANGEROUS, NOT JUST ANNOYING

A model opens `<parameter=content>`, pastes a whole Python file, and often **never closes the tag**. The parser must decide where the value ends.

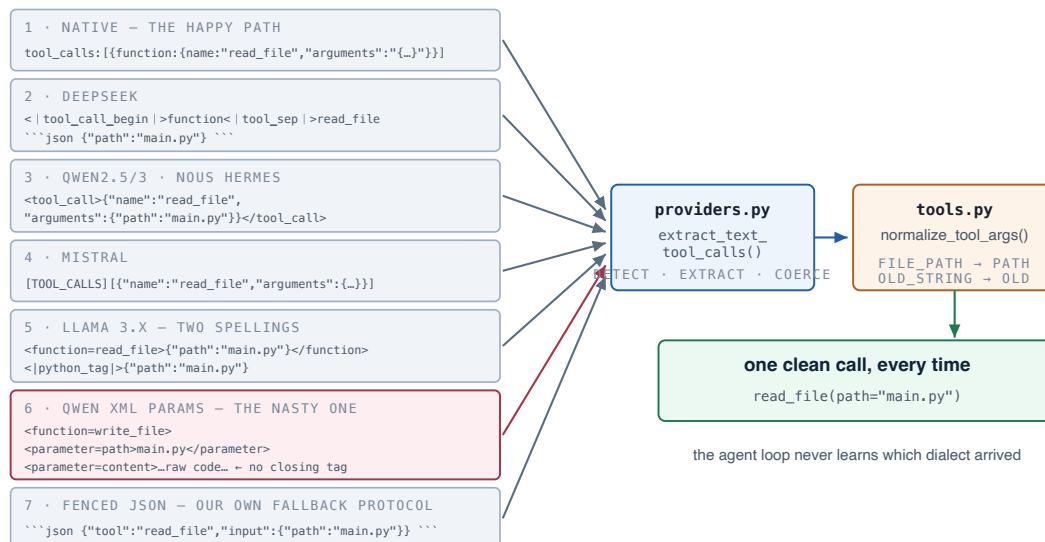
Too early: half the file is lost. Too late: **the next request's markup gets written into the user's source file**. That actually happened.

TWO RULES THAT KEEP IT SAFE

- **Two tool-shaped blocks in one reply → run neither.**
Running just the first silently drops the second and the model never learns it vanished.
- **A block inside prose or a `<think>` section is ignored.**
Otherwise a model *explaining* `rm -rf` would run it.

11 Seven dialects in, one shape out

PROVIDERS.PY → TOOLS.PY



Even after parsing, the option names are wrong

Local models learned from other tools' documentation, so they use *those* names. The fix is one lookup table — and its comments are the interesting part:

```
llmcode/tools.py — _ARG_ALIASES

"read_file": {"path": ("file_path", "filename", "target_file", ...)},
"write_file": {"path": ("file_path", ...),
# deliberately NO new_string/new_str here — those are edit_file's
# `new` key; mapping them to write_file content would turn a
# mis-targeted edit into a DESTRUCTIVE whole-file overwrite.
"content": ("file_text", "text", "body", "code", ...)},
"run_bash": {"command": ("cmd", "shell", "script", ...)},
```

A guessing table is dangerous. One plausible alias was left out **because the wrong guess would overwrite a whole file with a two-line fragment.**

The rule that makes guessing safe

```
cur = args.get(canonical)
if cur not in (None, ""): continue # model gave a value — it wins
if cur == "": continue # "" means empty file — keep it
for syn in syms: # else: fill the blank
    if args.get(syn): out[canonical] = args[syn]; break
```

An alias can **only fill a blank**, never overrule what the model said. And the dict is only copied if something actually changed — so the arguments you approved in the confirmation prompt are exactly the ones that run.

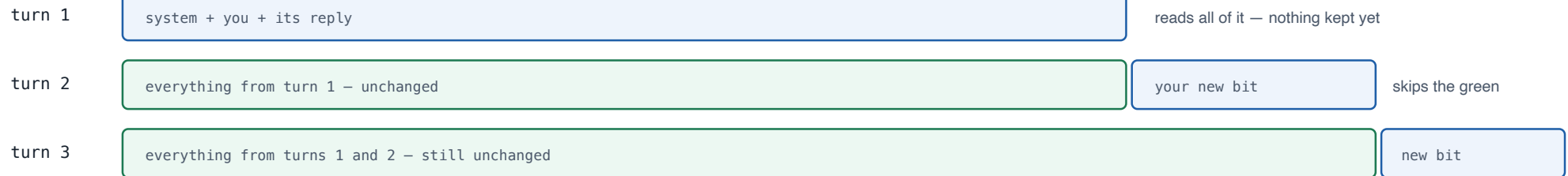
BE GENEROUS READING, STRICT ACTING

Accept seven formats, wrong key names, missing closing tags — but **refuse to act when the input is ambiguous.**

Misreading costs one wasted round. Acting on a misreading costs the user's files.

WHAT THE SERVER KEEPS

As it reads your message it builds up working state for every word. It **keeps that state** after answering.
Next time, it compares the start of your new message with what it kept — and reuses however much still matches, **exactly**.



The green part grows every turn, and every turn it is free. This is the whole trick.

AND HOW YOU THROW IT ALL AWAY



From the insert onwards nothing lines up any more, so all of it is read again — even though not one word changed.

SO THE RULE IS: ONLY EVER ADD TO THE END

Never insert. Never reorder. Never edit an earlier entry.
Text-only messages stay plain strings; tool descriptions are built once and reused; a summary that saved too little is rolled back.

BUT WE STILL NEED TO ADD THINGS

Recalled memories and per-turn hints go on a **copy** of the list, at the very end, on the first pass only.
`req_messages = self.messages + extras`

WHY IT MATTERS HERE

Most of your waiting is the model **reading** the message, not writing the reply. On a laptop that reading is the expensive part — and the server will skip it for free if you let it.

WHAT IT IS

Cache-augmented generation. RAG makes the message *smarter*; CAG makes it *cheaper*. It costs no extra code — only the discipline to never disturb the start of the list.

HOW WE KEEP IT

The stored list is **append-only**. Anything we want to add for one turn goes on a copy, at the very end. The list on disk and the list in memory are never edited in the middle.

What the server remembers

When it reads your list, it keeps the work it did. If your **next** request starts with *exactly the same entries, in the same order*, it reuses that work and starts where the new text begins. Most of your waiting is that re-reading — skipping it is free.

So what breaks it?

Changing anything that isn't at the end. Insert one entry in the middle and everything after it has shifted position — so all of it must be read again, even though not one word of it changed.

Which is a problem, because we *do* need to add things

Every turn we attach recalled memories (box 7) and sometimes a hint. The obvious place is near the question — in the middle. That would cost the cache every turn.

llmcode/agent.py — the whole solution, four lines

```
if _iter == 0:                                # first pass of this turn only
    extras = [b for b in (mem_block, hint_block) if b]
    req_messages = self.messages + extras # ← a COPY, extras LAST
else:
    req_messages = self.messages           # later passes: unchanged
```

✓ ADD IT TO THE END — WHAT WE DO



✗ SQUEEZE IT INTO THE MIDDLE — THE TEMPTING MISTAKE



One message added near the front throws away the saved work for everything after it — even though not one of those words changed. Where it sits matters, not just what it says.

THREE WINS FROM THAT ONE +

- `self.messages` is **never modified**, so its prefix stays byte-stable and the cache keeps paying.
- The extras are **never saved to disk**, because they were never in the stored list.
- Summarising can never mangle them, because they aren't there to be summarised.

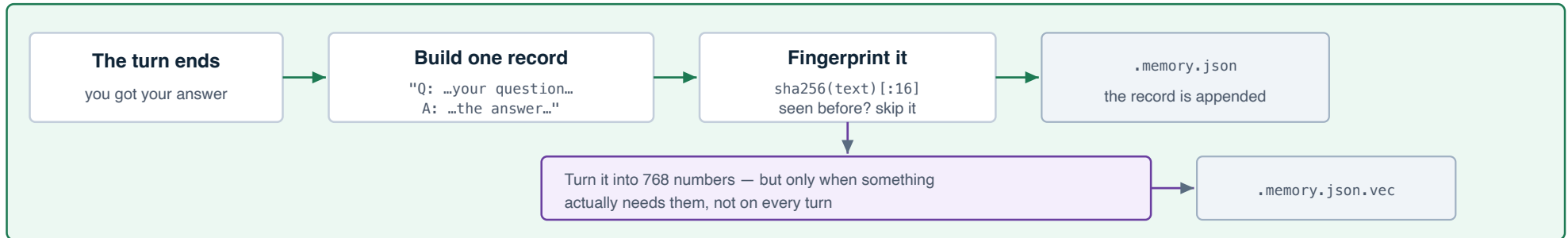
THE SAME RULE, THREE MORE PLACES

- A message with no image keeps `content` as a plain **string**; attaching an image makes it a list, which is a different prefix.
- The tool descriptions are built once and reused, so their key order can't drift.
- A summary that didn't save enough is **rolled back** (previous slide) — the code would rather carry extra words than disturb the front of the list.

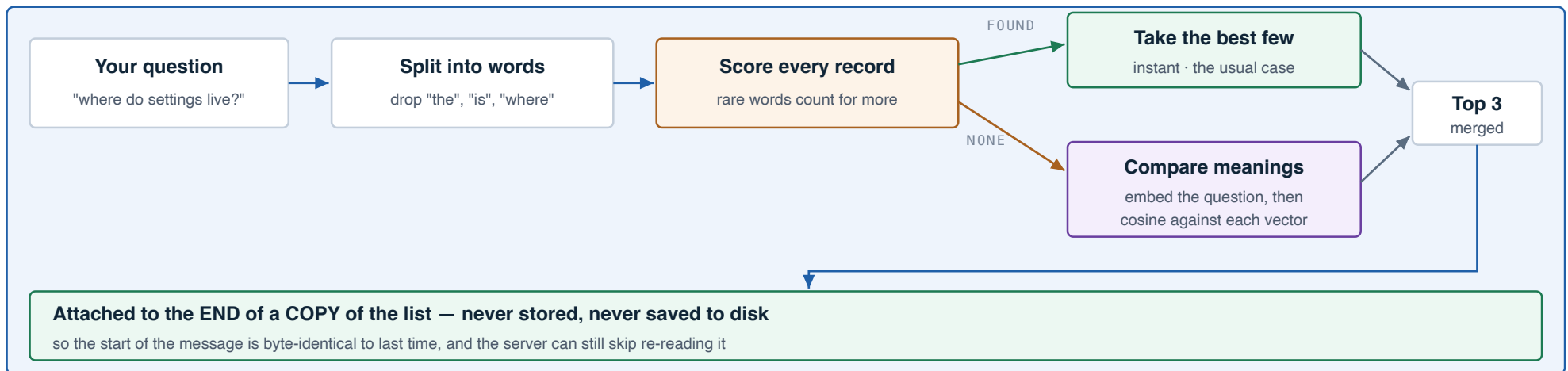
14 Looking things up, end to end

RAG · 1 OF 5

WRITING — ONCE, AT THE END OF EVERY TURN



READING — ONCE PER TURN, BEFORE THE LOOP STARTS



TWO PATHS, DIFFERENT COSTS

Writing happens **once per turn** and is nearly free. Reading happens **once per turn** too — never inside the loop, because searching on every pass would undo the speed it buys.

THE GATE IN THE MIDDLE

The slow half only runs when the fast half scored **exactly zero**. That is why filler words are stripped: it makes zero mean *nothing in common*.

WHERE IT ENDS UP

On a **copy** of the list, at the very end. Never written to the session file — so recalled text can never come back as if you had said it.

What gets stored, and when

At the end of every successful turn, **one record** is added — the question and the answer, glued together:

llmcode/agent.py — right before the turn returns

```
self.memory.add(
    text = f"Q: {user_text}\nA: {final_text}",
    summary = self._cheap_summary(final_text),
)
```

What one record looks like

llmcode/memory.py — MemoryRecord

```
{
  "id": "r17", # position when added
  "text": "Q: where is config stored?\nA: ~/.llmcode/...",
  "summary": "config lives in ~/.llmcode",
  "content_hash": "a7f3c91b2e4d8006", # sha256(text)[:16]
  "ts": "2026-09-18T09:14:02Z"
}
```

The whole file is a JSON list of those, at `~/.llmcode/sessions/<proj>.memory.json`.

WHY CONTENT_HASH EARNS ITS PLACE

It does three jobs at once: it **stops duplicates** being added, it is the **key** under which that record's vector is stored, and it means the same text is **never embedded twice** — even across restarts. Name things by a fingerprint of their content and caching becomes almost free.

How a record gets back into the conversation

Once per turn, **before** the loop starts — never inside it, because searching costs time and doing it every pass would undo the speed it buys:

llmcode/agent.py — run(), before the loop

```
if self._memory_active() and self.memory.records \
    and turn_intent != "trivial": # "thanks" skips it
    try:
        records = self.memory.retrieve(user_text, provider, ...)
    except Exception:
        records = [] # never break a turn
    mem_block = {"role": "user", "content": render(records),
                 "_memory": True} # ← tagged, not stored
```

That block is appended to a **copy** of the list (box 6), so it is sent but never kept. The `_memory` tag also means `session.py` strips it before saving — recalled text never gets re-saved as if it were new conversation.

Three design choices worth copying

- **Role is "user", not "system"**. A system entry arriving *after* tool results is rejected by some strict chat templates. A user entry is accepted anywhere.
- **Only on the first pass**. Re-sending it on every tool round wastes words and breaks the cacheable tail.
- **Failure is swallowed**. Recall is a bonus; losing it must never cost the answer.

First, every record is turned into a bag of words

```
_tokenize("Where is the config stored?")
→ lowercase → split on [a-z0-9]+ → drop filler words
→ ["config", "stored"]      # "where", "is", "the" dropped
```

The **same** function runs on records and on your question — that symmetry is the only reason they ever match.

Then one index is built for the whole store

```
llmcode/memory.py - _BM25Index, built once, reused for every search
```

```
tfs      : list[dict[str,int]] # per record: word → how many times
lengths  : list[int]          # per record: how many words
avgdl    : float              # average record length
df_map   : dict[str,int]      # word → how many records have it
n        : int                # how many records
```

Built in one pass over all records. It is thrown away and rebuilt only when the store changes — otherwise every search would re-count every word in every record, which is the difference between instant and a visible pause on a few thousand items.

DROPPING FILLER WORDS IS LOAD-BEARING

It looks like tidying. It is not. If a question and a record share *only* the word "the", the score comes out **above zero** — and the next slide only reaches for the slow meaning-search when the score is **exactly zero**.

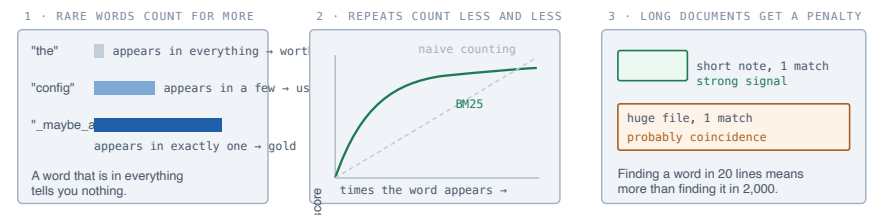
Our list deliberately **keeps** on , off , no , not : in a command-line tool those are values, not filler.

The scoring loop, in full

```
llmcode/memory.py - k1 = 1.5, b = 0.75
```

```
for term in q_terms:                # a repeated query word counts
    once
    f = tf.get(term, 0)              # times it's in THIS record
    if f == 0: continue
    n_q = df_map[term]               # how many records have it at all

    idf = log(1 + (n - n_q + .5) / (n_q + .5)) # ① rare =
    valuable
    denom = f + k1 * (1 - b + b * (dl / avgdl)) # ③ long =
    penalised
    s += idf * (f * (k1 + 1)) / denom          # ② repeats
    saturate
```



WHY NOT JUST COUNT MATCHING WORDS?

Because every record contains "the", long records would win by sheer size, and the tenth occurrence of a word would count as much as the second. Those three corrections **are** the formula — and it still beats fancier methods on code, where an exact function name matters more than general similarity.

17 Box 4 — the meaning search, and the gate that usually skips it

RAG · 4 OF 5 · MEANING SEARCH

What a vector actually is here

A record's text is sent to a second, small model (`nomic-embed-text`, served by the same LM Studio) which returns **768 floating-point numbers**. Texts about the same topic come back with similar numbers.

Comparing two of them — the whole function

```
llmcode/memory.py - cosine()

if not a or not b or len(a) != len(b): return 0.0
dot = na = nb = 0.0
for x, y in zip(a, b):
    dot += x * y      # how much they point the same way
    na += x * x       # length of a, squared
    nb += y * y       # length of b, squared
if na <= 0.0 or nb <= 0.0: return 0.0
return dot / sqrt(na * nb)    # 1.0 = same direction = same topic
```

We compare **direction**, not distance — length mostly reflects how wordy the text was. Note it **never raises**: garbage in returns `0.0`, which correctly means "no match", so one corrupt record can't take down a turn.

SEARCHING IS A LOOP, NOT A DATABASE

for each record: `cosine(query_vec, record_vec)`, keep the top few.
Over a few thousand records that is **milliseconds**. No index structure, no server.

The decision that made this fast

Embedding the question means **another round trip** to a different model — and on one GPU that can force your main model to be swapped out and back. Doing that on every turn would cost more than the search saves. So:

```
llmcode/memory.py - retrieve(), the entire policy

bm25_full = _bm25_score_index(query, self._bm25_index)
best_bm25 = bm25_full[0][1] if bm25_full else 0.0

embed_idx = []
if provider and (mode == "embed"
                  or (mode == "auto" and best_bm25 <= 0.0)):
    embed_idx = self._embed_rank(query, provider, pool_k)

ordered = bm25_idx if not embed_idx \
           else self._fuse(bm25_idx, embed_idx, pool_k)
```

YOUR QUESTION	WHAT RUNS	COST
"where is config.json?" shares real words	word search only	instant — the usual case
"where do settings live?" a paraphrase	word search scores 0 → meaning search → results merged	one round trip, only when needed
"thanks"	nothing at all	zero

THE SHAPE, WHICH RECURS FOUR TIMES IN THIS CODEBASE

Run the cheap check. Only pay for the expensive one when the cheap one found nothing. Here; in code search; in the optional re-ranking pass; and in the retry that lifts the reply-length cap.

IN RAM

```
{
  "a7f3...": array('f', [0.13, -0.88, 0.04, ...]),
  "b0c1...": array('f', [0.52, 0.19, -0.7, ...])
}
```

4 bytes per number — not Python's bulky floats

↓ SAVE

ON DISK — TWO FILES

.memory.json
the text, plus a fingerprint of the numbers
readable, and small

.memory.json.vec
just the raw numbers, back to back
all the same size — no labels, no punctuation

THE .VEC BYTE LAYOUT

vector 0 · 3072 B vector 1 · 3072 B vector 2 · 3072 B ...

same order as the list in the other file, so number 5 is easy to find

WHAT IF THE POWER CUTS HALFWAY?

Two files can disagree. If we save one and not the other, one says 200 items and the other holds 140 — and every item quietly matches the **WRONG** numbers.

Fix: save a fingerprint of the numbers file inside the other file. Check it on load.

If it doesn't match → throw the numbers away and rebuild them. Slow once, never wrong.

Why not just put them in the JSON?

768 numbers per record, written as decimal text, is roughly **9 KB of digits per record** — and it has to be parsed back one number at a time on every launch. A few thousand records makes the file unopenable and startup slow.

So: numbers in their own file, as raw bytes

In memory each vector is an `array('f')` — **4 bytes per number**, packed, instead of the ~32 bytes an ordinary Python float costs. Saving is then just writing those bytes straight out, end to end, with **no separators and no labels**.

Finding vector number 5 is arithmetic, not parsing:

```
stride = dim * 4           # 768 × 4 = 3072 bytes each
offset = i * stride        # vector i starts exactly here
```

The real engineering problem: two files can disagree

The records live in the `.json`; their numbers live in the `.vec`. If the process dies between writing one and the other, the JSON might claim 200 records while the `.vec` holds 140 — and then **every record quietly pairs with the wrong numbers**. No error. Just wrong answers, forever.

THE FIX: MAKE ONE FILE PROVE IT MATCHES THE OTHER

When saving, we fingerprint the exact bytes of the `.vec` and store that fingerprint **inside the JSON** as `vector_sig`.

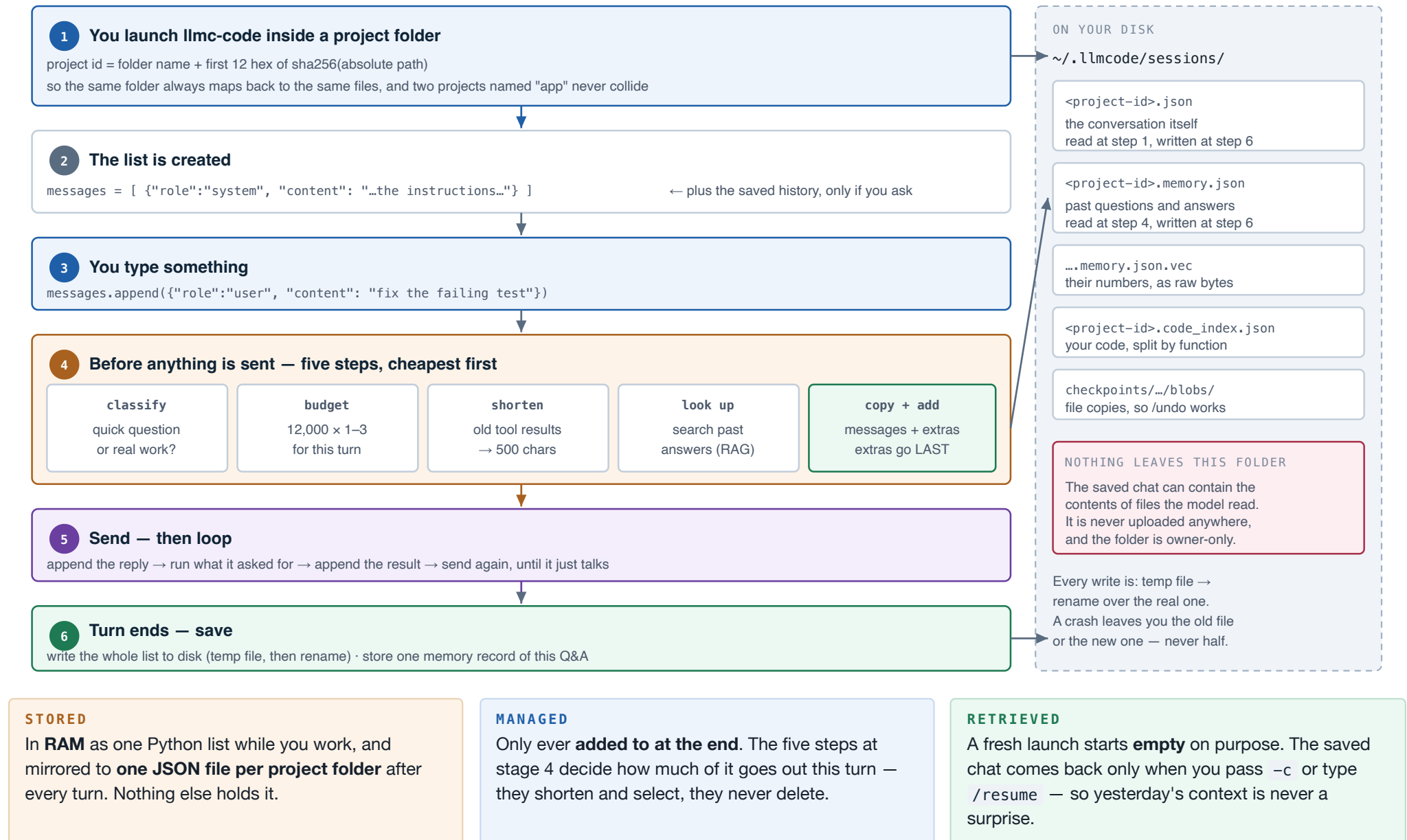
On load we re-fingerprint the file and compare. If it doesn't match — or the file is short, or missing — we **treat the vectors as empty and re-embed**. Slow once; never silently wrong.

WHEN THIS SIMPLE APPROACH IS THE RIGHT ONE

Thousands of items, one machine, one user — a loop over 4,000 comparisons is milliseconds. You need a real vector database at *millions* of items, or when several programs share one index. **Not before that.**

19 The life of a conversation — stored, managed, retrieved

CONTEXT · 1 OF 4 · THE WHOLE LIFECYCLE



Where is "the context" stored?

It is **not** a separate thing. The context is `agent.messages` — the list from slide 3, in RAM, mirrored to `~/llmcode/sessions/<proj>.json` after each turn.

"Managing context" means **rewriting strings inside that list** before we send it.

Why not just send the whole list every time?

Because the model has no memory, so we *do* resend it all — and that costs three different things:

COST	WHAT HAPPENS
Time	It reads every word before writing one. Reading time grows with the list; typing speed doesn't. So it just sits there longer.
Space	A local model holds a fixed amount. Past that, the oldest entries fall off silently — possibly the instruction that mattered.
Accuracy	Your question gets buried in stale tool output and it starts answering about a file it read six steps ago.

MEASURED, IN THIS PROJECT

"what happened?" — four words — shipped **55,000 words** of list and took **~20 minutes**. Over half of it was tool results from finished steps.

Step 1: how do we know how big it is?

We don't use a real tokenizer — that's a large download per model. We use a deliberately crude estimate over the whole list:

llmcode/tokens.py — the entire module

```
def estimate_text_tokens(s: str) -> int:
    return len(s) // 4          # characters ÷ 4. That's it.
```

It is wrong by roughly 15%, and that is fine — **every limit it feeds is a number we chose ourselves**. It is also kept as a **running total**: when a message is appended we add just that message's cost, instead of re-measuring the whole list. Re-measuring each time made turn 20 cost twenty times turn 1.

Step 2: how big is this turn allowed to be?

A budget is computed **per message**, before anything is sent:

llmcode/agent.py — `_compute_turn_budget()`

```
base = 12_000                                # the configured default
base = base * request_weight(text)           # 1.0 ... 3.0
# +0.5 if over 400 chars   +0.5 if over 1200 chars
# +1.5 if it says audit / refactor / migrate / codebase...
if classify_request(text) == "trivial":
    base = min(base, 12_000 // 2)             # "thanks" gets half
budget = max(4_000, base)                     # never starve a turn
```

THE BUDGET IS A TARGET, NOT A KNIFE

Nothing is deleted here. This just sets the number that the next two slides — shortening and summarising — try to get the list *under*.

The idea in one sentence

Walk the list, find tool results from **turns that are already finished**, and replace each one's `content` string with its **first 500 characters plus a note**. Nothing is deleted — the entry stays exactly where it was.

```
llmcode/agent.py - _trim_stale_tool_outputs(), the core

# 1. find the LAST user message - that is the boundary
user_idx = [i for i, m in enumerate(self.messages)
             if m["role"] == "user"]
boundary = user_idx[-1]

# 2. walk everything BEFORE it (skip [0], the system message)
for m in self.messages[1:boundary]:
    if m["role"] == "tool":
        c = m["content"]
        if len(c) > 500 and _TRIM_MARKER not in c:
            m["content"] = (c[:500] +
                           f"\n...[{len(c)-500}] chars <ctx-trimmed> - "
                           "older tool output; re-run or re-read it")
            mutated = True
```

The same pass also reaches **inside** old `write_file` and `edit_file` requests and cuts their `content` / `old` / `new` fields to **160 characters** — a whole file you wrote six steps ago is still sitting in that request, and it is usually the single biggest thing in the list.

Four engineering details that make it safe

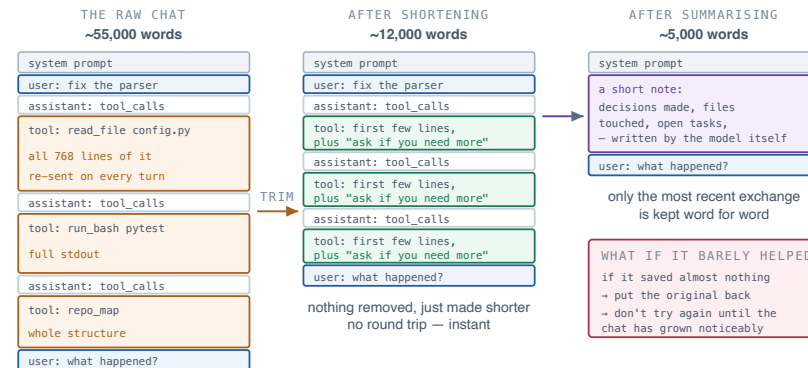
DETAIL	WHY IT HAS TO BE THAT WAY
The boundary is the last user message	Everything after it belongs to the turn happening <i>now</i> . A follow-up like "and the other file?" needs that in full. Only finished turns are touched.
Only <code>content</code> changes never the list itself	No entry is removed, none reordered, no <code>tool_call_id</code> touched. So the pairing rule from slide 3 cannot break. The list is the same length before and after.
The marker check <code><ctx-trimmed></code> not in <code>c</code>	This runs on every turn over the same entries. Without the check it would cut the already-cut note again next turn, and again, eating its own explanation until nothing was left. The marker is a deliberately weird string so real file content can never contain it by accident.
The note says what to do	Not a silent truncation. The model reads " <i>older tool output — re-run or re-read it</i> " and goes and re-reads the file. A silent cut makes it invent the missing part instead.

THE ONE SUBTLE BUG IN HERE

The function reports how much it saved as `saved_chars // 4`. Integer division can round a **real change down to 0**. So the caller must decide whether to invalidate the size cache using the separate `mutated` flag — **never the returned number**. A cache invalidated by a lossy signal eventually lies to you.

22 Box 3 — and when shortening isn't enough: summarising

CONTEXT · 4 OF 4 ·
SUMMARISING



What it does to the list

Takes everything except the **last exchange**, asks the model to summarise it, and **replaces that whole run of entries** with one summary entry — dozens of entries become one. It costs an extra round trip, so it is the last resort.

```
llmcode/agent.py - _maybe_auto_compact()

est = self._estimate_tokens_cached()
if est <= self.context_soft_limit: return # under budget, nothing to do
if est <= self._compact_floor: return # already tried at this size

snapshot = self.messages # ← keep the original
before, after = self.compact(keep_turns=1) # ask the model

if before - after >= max(128, before // 33): # saved ≥3%? keep it
    self._compact_floor = 0
else:
    self.messages = snapshot # ← put it back
    self._compact_floor = int(est * 1.1) + 1 # don't retry until +10%
```

Why every line of that guard exists

LINE	THE FAILURE IT PREVENTS
<code>snapshot first</code>	A small model sometimes writes a summary as long as the original . You need the original back.
<code>≥ 3% or 128 tokens</code>	A tiny saving isn't worth rewriting the list — which also throws away the server's cached work (box 6).
<code>_compact_floor</code>	Without it, a summary that didn't help would be re-attempted every single turn , paying a round trip each time for nothing.
<code>keep_turns=1</code>	Was 2. With 2, the second-to-last turn's bulk stayed in the kept tail and never got summarised — heavy sessions barely shrank.

SUMMARISING IS NOT DELETING

Deleting old entries is one line, and loses every decision in them. The summary prompt asks specifically for **decisions made, files touched, open tasks and key facts**.

Your code is stored the same way memories are

llmcode/code_index.py — CodeChunk

```
{
  "path": "llmcode/agent.py", # relative to the project
  "start_line": 2696, # 1-based, inclusive
  "end_line": 2784,
  "text": "def _trim_stale_tool_outputs(self)...",
  "content_hash": "9c02ab71f4e5d833"} # same trick as memory
```

Saved as a JSON list at `~/.llmcode/sessions/<proj>.code_index.json`, with its vectors in a `.vec` sidebar — the **same two-file pattern** as the previous slide.

The chunk boundary is the important decision

Chunks are cut **at function and class boundaries**, not every 50 lines. A result that starts halfway through a function teaches the model nothing; where you cut should be where the *meaning* ends.

Re-indexing is incremental

On each search we compare each file's **modification time and size** against what the index recorded. Only changed files are re-chunked; the unchanged ones keep their chunks *and their vectors*, because the hash is unchanged.

WHY WORD SEARCH IS THE DEFAULT HERE

Searching code usually means searching for a **name you already know**. Building a meaning-index over a whole repo takes minutes on a laptop; the word search takes milliseconds and usually wins anyway. Meaning search is opt-in via `/codeembed`.

The project map: which files even matter?

A flat list of 300 filenames is useless to the model. So we rank them by **how many other files depend on them** — the same idea as ranking web pages by who links to them.

```
# 1. build a graph: which file imports which
# 2. run PageRank over it → a score per file
# 3. sort by score: config.py high, one-off script low
```

Then it must be made to fit a size limit

You can't know how many entries fit until you render them. So: render, measure with `chars // 4`, adjust, repeat — **halving the range each time**, like guessing a number between 1 and 1000. About ten tries lands exactly on the budget.

THE PERFORMANCE BUG THAT HID IN HERE

The first PageRank re-scanned **every connection for every file** on each iteration. On a real repo that made `repo_map` the slowest tool we had — the tool whose entire job was to save time.

Rewriting it to walk each connection once turned it from noticeable into instant.

THE PRINCIPLE BEHIND THIS WHOLE SLIDE

Don't give the model more to read. Give it better ways to find what it needs.

A good search tool beats a bigger memory, and costs far less.

24 Sub-agents — how the context stays small

ORCHESTRATION.PY · AUDIT.PY

A sub-agent is just another Agent object

No new class, no new loop. `spawn_agent` is a **tool** like any other — and what it does is construct a second `Agent` :

llmcode/orchestration.py — what the tool actually does

```
sub = Agent(
    provider = self.provider,          # same model
    messages = [{"role": "system",      # FRESH list —
                  "content": ROLE_PROMPTS[role]}], # empty history
    registry = subset_of(REGISTRY, ROLE_TOOLS[role]),
    render_answer = False,             # parent prints, not it
)
answer = sub.run(task)                 # runs its own full loop
return {"ok": True, "result": answer}  # ← a normal tool result
```

The helper's own `messages` list — possibly 40,000 words of files it read — is a **local variable**. When `_spawn()` returns, it is garbage collected. Only the returned string ever reaches the parent's list.

THE NUMBERS

Helper reads 20 files → **~40,000 words in its list**.

Parent receives **~15 lines**, as one `role: "tool"` entry.

The parent's context is protected **by construction**, not by the model being careful.

Roles are just different dict subsets

ROLE	REGISTRY CONTAINS	JOB
orchestrator	everything + <code>spawn_agent</code>	plan, delegate, check
explorer	<code>read_file</code> <code>glob</code> <code>grep</code> <code>repo_map</code>	investigate, report
coder	the full set	make one change, verify it
reviewer	read-only	criticise before it's called done

SECURITY BY ABSENCE, NOT BY INSTRUCTION

An explorer has **no `write_file` key in its registry dict**. No wording in a file it reads can talk it into writing, because there is nothing to call. Take the capability away — don't ask the model nicely.

The same trick, scaled up: `/audit`

Reviewing a whole project doesn't fit in a local model's head. So:

1. pack files into chunks of **~1500 lines** # max 250 files, 24 chunks
2. **for** each chunk: a fresh Agent reviews **only that chunk**
3. collect the short findings (not the file contents)
4. one more Agent, **with no tools at all**, merges them

And when it hits those caps it **says so loudly** rather than reporting "clean" — a silent partial review is worse than no review.

Nine ways it knows to stop

WHAT IT WATCHES FOR	THE LIMIT
Going round and round	80 times, then stop
Requests we can't read at all	3 bad rounds
Asking for the exact same thing again	4 repeats
It thought, but said nothing	2 retries
The reply got cut off	once per round
The request was malformed	once each
You pressed Ctrl-C	checked every round
It's reading far too much	nudged once a turn

A SAFETY NET THAT CATCHES HEALTHY BEHAVIOUR IS A BUG

Our first version counted **any** bad request and shut down sessions that were working fine. A model that makes one good request and one bad one *is still getting somewhere*.

Now it only counts when **everything** in that round failed — and a single good request resets the count.

Treat the model like any untrusted input

- **It asks you first** before writing, editing, or running anything.
- **Workspace confinement** on the *file* tools (read / write / edit / glob / grep / repo_map) — paths resolving outside the launch directory are refused.
- **Your secrets are stripped out** — commands it runs never inherit your API keys or cloud credentials.
- **Web fetching can't reach inside your network** — private addresses are refused, and re-checked on *every redirect*, so it can't be talked into it halfway.
- **Undo checkpoints** — files are snapshotted before every edit.
- **Small mistakes fix themselves** — a wrong path is corrected and retried, a limited number of times.

--PRIVATE : AN ENFORCED OFFLINE LOCKDOWN

Not a promise — an actual mechanism. The model address **must** be your own machine. Web fetching is **removed entirely**, so the model can't even ask. Commands run with the network blocked, and if that blocking isn't available, they **refuse to run at all** rather than run unprotected.

THE MOST VALUABLE BUG WE EVER FIXED

Our own docs said the tool couldn't write outside your project folder. For shell commands that was simply **not true** — it is a raw shell, with no path check in it at all. What actually protects you there is **the confirmation prompt**.

A safety claim that isn't true is worse than no claim, because people relax *because* of it. Say which mechanism enforces a guarantee — or don't make it.

WHY

A cloud tool never thinks about hardware. On **your** laptop, an hour of flat-out work means screaming fans — then the machine **deliberately slows itself down** to cool off. The longer you work, the worse it gets.

WHAT

Two features **no cloud tool has**:

- a forced **one-minute break** after roughly ten minutes of solid work
- **shorter replies and small pauses** between turns

HOW

The break happens at the **top of the loop**, where nothing is half-finished. Both are on by default, and both switch themselves off when output is piped — so scripts and tests never sit there waiting.

WHY THE TOP OF THE LOOP

Pausing in the middle of a step would leave a request with no matching result sitting in the conversation — and most servers **reject the whole thing** when they see that.

The position isn't arbitrary — it makes breaking **impossible**, not just unlikely. Ctrl-C is handled at the same spot, for the same reason.

THE BUG THAT GENTLE MODE CAUSED

We capped how much the model could write, to keep the GPU cool. On a normal answer, hitting the cap just cuts off a sentence.

But when it *writes a file*, the file content travels **inside the request itself**. Cut that off and the request is incomplete — the model thought it had written the file, the user thought so too, and **nothing reached the disk**.

The fix, in three parts: the cap is now **skipped entirely whenever tools are on the table**; the default was raised **1,024 → 4,096**; and anything still cut off is retried **once** with no cap at all.

And the twist: cutting off then retrying meant the model wrote the answer *twice* — which produced **more** heat than the cap ever saved.

THE LESSON

A limit you set for one reason will be hit by something you never thought about. Whenever you add a cap, ask what *e/else* travels down that path.

THE HONEST REFRAME

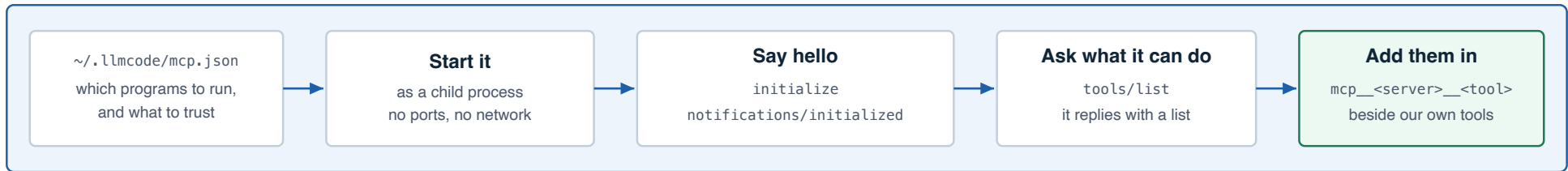
We spent a long time trying to make it *type* faster. Typing speed was **never** the problem — it sat near 95 words a second throughout. The real pain was the **waiting**.

Nobody experiences "words per second". They experience **"I pressed Enter, and then I waited."** That is the number to improve.

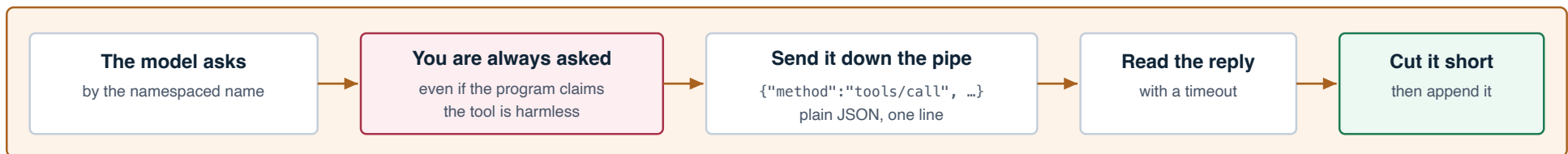
27 MCP — plugging in tools we didn't write

MCP · 1 OF 2 · START, CALL, FAIL

1 · WHEN LLMC-CODE STARTS



2 · WHEN THE MODEL WANTS TO USE ONE



3 · WHEN SOMETHING GOES WRONG, OR YOU QUIT

A program that does not answer in time is treated as poisoned

the child is killed and its tools are dropped — so a late reply can never be mistaken for the answer to a later question
on exit: close its input, ask it to stop, then force it. One that fails to start is skipped with a warning, never crashing the CLI.

WHY

Our eleven tools can't cover everything. Others have already built tools for databases, note vaults, ticket systems. **MCP is the agreed way to talk to any of them**, so we don't write a connector each time.

WHAT IT COSTS US

741 lines and no new libraries. It is a small program started as a child process, spoken to in plain JSON down its input and output — no ports, no network.

HOW WE STAY SAFE

A program can claim its tool is harmless. That claim is **ignored** unless you personally trusted that program. Names are checked so one can't impersonate another.

28 Other people's tools, and testing with a fake model

MCP · 2 OF 2 · AND TESTING WITH A FAKE MODEL

Plugging in tools we didn't write

WHY

Our eleven tools can't cover everything. Other people have already built tools for databases, note vaults, ticket systems. **MCP** is an agreed way for any AI tool to talk to any of them, so nobody has to write a special connector each time.

WHAT

741 lines, no new libraries. Each tool program is started as a small background process, and its abilities are added to the model's list next to our own.

HOW

Start it, ask what it can do, add those to the list. One that fails to start is **skipped with a warning** — it never takes the whole program down with it.

NEVER TRUST A SERVER'S OWN HINTS

A tool program can declare "this one is harmless, no need to ask". But a *bad* one could say that about something destructive, to get it run automatically.

So that claim is ignored unless **you** have said you trust that particular program.

The decision to trust is always yours, never the tool's.

Also: names are checked so one program can't impersonate another, duplicates are skipped rather than silently overwriting, and a program that stops responding is **shut down** — so a late reply can never be mistaken for the answer to a different question.

The single best decision in this project

We never let the code call the model directly. It always goes through a **swappable piece**. That let us write a second one: a **fake model that follows a script**.

real or fake, both produce exactly these three things

```
{"type": "text",      "text": <incremental delta>}
{"type": "tool_call", "id", "name", "arguments": <dict>}
{"type": "done",      "finish_reason", "output_tokens"} # exactly
one, last
```

1,355

tests

0

of them need
a GPU or internet

3

dependencies
openai · rich · prompt_toolkit

WHY THIS MATTERS MORE THAN IT SOUNDS

Nearly every bug here is a **loop bug**: a request with no matching result, a counter that never resets, messages added in the wrong order.

You cannot chase those with a real model — it gives a different answer every time, and slowly. Against a script they become **ordinary tests that run in milliseconds**.

DO THIS ON DAY ONE

Adding this swappable piece later, once twelve different places all call the model directly, is miserable work. Done at the start, it costs ten minutes.

What we built, in one line each

THE PART	THE IDEA BEHIND IT
The loop	One Python list. Append the reply, run what it asked for, append the result, repeat.
Tools	A 5-field dataclass in one dict. Cap output at the source, return errors as data, never raise.
Dialect parsing	Seven parsers in, three event types out. Refuse to act when the input is ambiguous.
Context mgmt	chars÷4 to measure, a per-message budget, shorten old entries in place, summarise as a last resort.
CAG	Never modify the stored list — send <code>self.messages + extras</code> , a copy.
RAG	One JSON file of records + a word index. Meaning search only when the word score is exactly 0.
Vector store	4 bytes per number in a <code>.vec</code> file, plus a checksum stored in the JSON.
Sub-agents	A second Agent with a fresh list and a smaller registry. Its reading is thrown away.
Hardware	A pause at the top of the loop, where nothing is half-written.
Safety	Counters with real numbers, a confirmation gate, a scrubbed environment, file copies before edits.

IF YOU REMEMBER ONE THING

The model is not the program. Around 95% of this code is the machinery *around* the model — and that machinery is where the quality actually comes from.

Start here: a working assistant in 20 lines

Everything here grew out of the loop on slide 6. Type it, run it, then grow it — each exercise rebuilds one slide:

#	EXERCISE	WHAT YOU'LL DISCOVER
1	Add writing and editing files	Edits fail over invisible spacing. You'll invent a forgiving match.
2	Print the conversation size after each turn	Watch it explode after three file reads. That's the whole problem.
3	Summarise old turns once it gets big	And why simply deleting them loses your decisions.
4	Build a word search over your code	It beats meaning-search on code far more often than you'd think.
5	Add meaning-search, but only when the word search finds nothing	Cheap first, expensive only when needed.
6	Copy files before editing; add an undo	People approve far more changes once undo exists.
7	Send a read-only helper to do the exploring	Its reading gets thrown away; yours stays clean.
8	Point it at a model that asks in a different format	Reality. You'll write your second reader, then your third.

FINAL ADVICE

Build the loop first, and let it be ugly. Then spend your time on **better tools** and **keeping the conversation small**. That is where almost all the difference between a frustrating assistant and a genuinely useful one actually lives.